

IST602 Lecture 5 — Client-Side Programming (2025–2026) Detailed Summary

1) Lecture intent and progression

This lecture introduces JavaScript as the browser-side programming layer for adding behavior to web pages. The flow is practical and incremental:

1. Script basics and browser dialogs,
2. Control structures,
3. Functions and events,
4. Scope,
5. Arrays and common array operations,
6. Built-in objects (Math, String, Date, document, window).

It is fundamentally a “from syntax to interactivity” lecture.

2) Script fundamentals and browser runtime model

2.1 `<script>` and output primitives

- JavaScript is embedded via `<script>`.
- `document.write` / `document.writeln` demonstrate immediate markup output into page rendering flow.

2.2 Window interaction methods

- `window.alert(...)` for modal messages.
- `window.prompt(...)` for user input.
- Input from `prompt` is string-based and often requires conversion.

2.3 Type conversion basics

- `parseInt(...)` / `parseFloat(...)` convert user input strings to numeric values before arithmetic.
 - Reinforces that unchecked string addition can produce concatenation instead of numeric results.
-

3) Control statements and iterative logic

The lecture mirrors common imperative patterns:

- `while`,
- `for`,
- `do...while`,
- sentinel-controlled loops.

Key learning outcomes:

- Counter-controlled vs sentinel-controlled iteration.
- Increment patterns (`++counter`).
- Computing aggregates (sum, average).
- Loop-driven dynamic HTML generation.

The class average and font-size examples illustrate logic-to-UI coupling through repeated output.

4) Formatting and generated content

Examples combine JavaScript and CSS to build table output and style rows:

- Compound-interest computation using `Math.pow`.
- Row styling via class assignment (e.g., odd/even visuals).
- Numeric display formatting with `toFixed`.

This teaches that JavaScript can both **compute** and **render** presentable results in one pass.

5) Functions and modularization

5.1 Function composition

The lecture demonstrates decomposition:

- `rollDice` coordinates behavior,
- `setImage` handles image assignment,
- helper functions isolate repeated logic.

5.2 Built-in + user-defined functions

- Built-ins (`Math.max`, `Math.random`, `Math.floor`) are combined with custom wrappers.
- Shows practical reuse over copy-paste logic.

5.3 Event-driven architecture

Core event pattern:

- Register listeners with `addEventListener`.
- Trigger behavior on events (e.g., `load`, `click`).
- Keep setup in initialization function (`start`).

This transitions students from linear script execution to browser event lifecycle thinking.

6) DOM interaction patterns

The lecture repeatedly uses:

- `document.getElementById(...)` for element retrieval,
- `setAttribute(...)` for dynamic attribute updates (e.g., image `src/alt`),
- `innerHTML` for injecting generated content blocks.

These operations demonstrate the DOM as an object graph where content and presentation are updated at runtime.

7) Case study: dice simulations and statistics

Two dice examples develop increasing sophistication:

1. **Single roll visualization**

- Button click updates four image elements with random dice faces.

2. **Repeated rolling with frequency table**

- Rolls many times per action,
- Tallies face frequencies using counters and `switch`,
- Computes percentages,
- Renders statistical summary table dynamically.

Conceptual outcomes:

- Random process simulation,
 - Accumulator/state management,
 - Mapping numerical state to UI,
 - Basic probability intuition through empirical frequencies.
-

8) Scope rules and variable visibility

The lecture formalizes:

- **Global scope:** identifiers declared outside functions.
- **Function/local scope:** parameters and local declarations.
- Name shadowing/hiding when local names match global names.

The scoping example clearly shows:

- local variables reinitialize per call,
- globals persist and can be mutated across calls.

This is foundational for avoiding side effects and debugging runtime behavior.

9) Arrays as dynamic object-based collections

9.1 Core properties

- Arrays are objects in JavaScript.
- Zero-based indexing.
- Dynamic sizing.
- Length available via `.length`.

9.2 Construction patterns

- `new Array(size)`,
- `new Array(...)` with initial values,
- Literal initializer `[ ... ]`,
- Sparse arrays with omitted slots.

9.3 Passing arrays to functions

- Arrays (objects) are effectively passed by reference semantics in examples.
- Primitive elements are passed by value.
- Demonstrates why function calls can mutate whole-array state.

9.4 Common methods

- `sort()`:
 - default string-based ordering,
 - comparator needed for numeric order.
 - `join(...)` for formatted output.
 - `indexOf(...)` for search.
-

10) Objects and built-ins

10.1 Object-oriented mindset

- Objects bundle data + behavior.
- Information hiding and interface-based interaction are introduced conceptually.

10.2 `Math` object

- Used for arithmetic utilities (`pow`, `sqrt`, random-related operations).

10.3 `String` object

Major method categories covered:

- Character access (`charAt`, `charCodeAt`, `fromCharCode`),
- Case conversion (`toLowerCase`, `toUpperCase`),
- Search (`indexOf`, `lastIndexOf`),
- Tokenization and substring extraction (`split`, `substring`).

10.4 `Date` object

- Multiple constructor forms (current date, epoch ms, explicit components).
- Local vs UTC representation methods.
- Getter and setter families for date/time parts.

10.5 `document` and `window`

- `document`: manipulate current page structure/content.

- `window`: browser window control (`open`, `close`, URL/location, state).
-

11) Architectural skills this lecture builds

1. Turning user input into validated numeric processing.
 2. Structuring scripts around reusable functions.
 3. Reacting to lifecycle and UI events.
 4. Maintaining state across interactions.
 5. Updating DOM elements safely and intentionally.
 6. Choosing the right built-in objects/methods for common tasks.
-

12) Common pitfalls surfaced by examples

- Forgetting string-to-number conversion after `prompt`.
 - Using `sort()` numerically without comparator.
 - Confusing global and local variables.
 - Over-reliance on `document.write` in modern page lifecycles.
 - Fragile dynamic HTML assembly without careful structure.
-

13) Exam/practical revision checklist

Be able to:

1. Explain `alert`, `prompt`, and `document.writeln` roles.
 2. Implement counter and sentinel loops.
 3. Write event wiring using `addEventListener`.
 4. Use `getElementById`, `innerHTML`, and attribute mutation.
 5. Explain scope, shadowing, and side effects.
 6. Pass arrays to functions and predict mutation behavior.
 7. Sort numbers correctly using comparator functions.
 8. Use key methods from `Math`, `String`, and `Date` objects.
-

14) One-line takeaway

This lecture turns JavaScript from syntax into practical browser programming by combining control flow, DOM manipulation, event handling, arrays, and built-in objects to create interactive, stateful pages.